

UNIVERSIDAD DE LAS FUERZAS ARMADAS / POLITÉCNICA

FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS
CARRERA DE INGENIERÍA EN SOFTWARE / TECNOLOGÍAS DE LA INFORMACIÓN

TAREA DE RECUPERACIÓN

TEMA DE INVESTIGACIÓN Y DESARROLLO TÉCNICO:

DISPOSITIVOS REALES Y VIRTUALES, CONSUMO DE APIS REST GENERACIÓN DE APK EN APLICACIONES MÓVILES

SUBTEMAS DE ESTUDIO:

- Módulo I: Dispositivos móviles reales, emuladores, simuladores y Android Virtual Device (AVD).
- Módulo II: Arquitectura REST, protocolo HTTP/HTTPS y cliente Axios con interceptores.
- Módulo III: Pipeline de compilación Android, optimización con zipalign y firmado apksigner.
- Módulo IV: Desarrollo de solución práctica modular y scripts de construcción y despliegue.

DOCENTE / TUTOR:

Ing. Luis Calo

ASIGNATURA:

Desarrollo de Aplicaciones Móviles

ESTUDIANTE / AUTOR:

Investigador y Desarrollador

CALIFICACIÓN Y PERÍODO:

Rúbrica: 10.0 / 10.0 Puntos | 2026

1. OBJETIVOS

OBJETIVO GENERAL

Analizar, contrastar y aplicar los fundamentos de entornos de prueba móviles (dispositivos físicos y virtuales), la integración de servicios web bajo arquitectura REST mediante Axios, y el proceso de construcción, optimización (zipalign) y firmado criptográfico de archivos APK para Android.

Objetivos Específicos

1. Analizar el funcionamiento interno, arquitectura, ventajas y desventajas de dispositivos reales, emuladores (QEMU/AVD) y simuladores.
2. Examinar la arquitectura RESTful y el protocolo HTTP/HTTPS, evaluando Axios frente a Fetch en interceptores, timeouts y errores.
3. Describir el pipeline de compilación de Android (bytecode DEX, D8/R8, AAPT2, Gradle) y el firmado con keytool, zipalign y apksigner.
4. Desarrollar una aplicación móvil funcional con cliente Axios modular, operaciones CRUD y scripts automatizados de compilación.

2. MARCO TEÓRICO

2.1. Dispositivos Reales y Dispositivos Virtuales en el Desarrollo Móvil

El desarrollo móvil requiere validar aplicaciones en múltiples combinaciones de hardware, arquitecturas de CPU (ARMv7, ARM64, x86_64), tamaños de pantalla y versiones de Android para mitigar la fragmentación del ecosistema.

2.1.1. Dispositivo Móvil Real (Hardware Físico)

- ¿Qué es? Es una unidad física comercial de smartphone o tablet con un System-on-Chip (SoC) basado en arquitectura ARM/ARM64, circuitos de radiofrecuencia (4G/5G, Wi-Fi, BLE, NFC) y sensores físicos (GPS, giroscopio, cámaras, biometría).
- ¿Cómo funciona? La app se ejecuta de forma nativa sobre el Android Runtime (ART) y los controladores de bajo nivel (HAL). La depuración se realiza mediante Android Debug Bridge (ADB) sobre cable USB o conexión inalámbrica TCP/IP (Wi-Fi).

VENTAJAS

- Fidelidad 100% en pruebas de usuario y gestos táctiles.
- Medición exacta de consumo de batería (mAh) y temperatura.
- Acceso directo a sensores físicos, biometría y Bluetooth real.

DESVENTAJAS

- Alto costo para adquirir diversidad de modelos y marcas.
- Desgaste acelerado de baterías en pruebas de laboratorio.
- Dificultad de escalabilidad en pipelines CI/CD automatizados.

2.1.2. Emuladores vs. Simuladores: Diferencias de Arquitectura

- Emulador (Hardware + Software): Modela tanto el hardware como el software del dispositivo objetivo. Emula las instrucciones de la CPU, registros y controladores mediante hipervisores y traducción binaria (ej. Android Virtual Device sobre QEMU/KVM). Ejecuta el kernel Linux real de Android.
- Simulador (Solo Software): Recrea únicamente las APIs y el comportamiento del sistema operativo a nivel software sobre el hardware y kernel del equipo host (ej. iOS Simulator en Xcode sobre macOS). No emula hardware a bajo nivel.

2.1.3. Android Virtual Device (AVD)

Es una configuración de emulador gestionada por el Android SDK que combina un perfil de hardware con una imagen de sistema (System Image x86_64/ARM64). Funciona sobre el motor QEMU con aceleración por hardware (KVM/HAXM) y aceleración por GPU passthrough.

Criterio Técnico	Dispositivo Real	Emulador AVD (QEMU)	Simulador (Software)
Nivel Virtualización	Nativo (Sin virtualizar)	Hardware y Software	Solo APIs / Software
Fidelidad de Pruebas	100% (Prueba de oro)	90% - 95% (Muy Alta)	70% - 80% (Media)
Consumo Host (PC)	Nulo (Usa su CPU/RAM)	Muy Alto (2-4GB RAM)	Bajo - Moderado
Sensores / Red	Reales y físicos	Simulados por software	Mocks / Limitados
Integración CI/CD	Compleja y costosa	Excelente (Headless)	Muy Rápida y fácil

VENTAJAS

- Permite emular múltiples versiones de Android (API 21 a 35+).
- Simulación de eventos: pérdida de red, llamadas y GPS falso.
- Ideal para pruebas automatizadas con Espresso y Appium en CI/CD.

DESVENTAJAS

- Elevado consumo de memoria RAM y CPU en la computadora host.
- No replica el estrangulamiento térmico ni capas de fabricantes.

2.2. APIs y Servicios Web para Aplicaciones Móviles

Las aplicaciones móviles modernas operan como clientes desacoplados que interactúan con servidores remotos mediante el protocolo HTTP/HTTPS e intercambian datos estructurados en formato JSON bajo principios REST.

2.2.1. Protocolo HTTP/HTTPS y Arquitectura REST

- Protocolo HTTP (RFC 7231): Modelo cliente-servidor con métodos semánticos:
 - GET: Consulta y obtención de recursos (Idempotente y seguro).
 - POST: Creación de nuevos registros o envío de transacciones.
 - PUT / PATCH: Actualización total o parcial de recursos.
 - DELETE: Eliminación de recursos en el servidor.
- Códigos de Estado: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Internal Server Error.
- Arquitectura REST: Sistema sin estado (Statelessness) donde cada solicitud incluye las credenciales necesarias y los recursos son identificados mediante URIs lógicas.

2.2.2. Axios para Consumo de Servicios en Aplicaciones Móviles

- ¿Qué es? Axios es un cliente HTTP isomórfico basado en Promesas ECMAScript diseñado para Node.js, navegadores y frameworks móviles (React Native / Capacitor).
- ¿Cómo funciona? Envuelve los adaptadores nativos de red y proporciona serialización automática de JSON, soporte para cancelación con AbortController, control de timeouts y una arquitectura de Interceptores de Petición y Respuesta.

Característica	Cliente Axios	Fetch API Nativa
Transformación JSON	Automática (Serializa y parsea data)	Manual (Requiere response.json())
Interceptores Request/Response	Nativos (Inyección de JWT centralizada)	No soportados (Requiere wrapper manual)
Manejo de Errores (4xx/5xx)	Rechaza la promesa automáticamente	Resuelve con éxito (Verificar res.ok)
Configuración de Timeouts	Soporte nativo (timeout: 10000ms)	Requiere AbortSignal.timeout() complejo
Protección XSRF / Headers	Integrada por defecto	Configuración manual

' VENTAJAS

- Patrón Interceptor para inyectar Bearer Tokens de forma global.
- Manejo explícito de timeouts para mitigar redes móviles inestables.
- Normalización limpia de errores de red vs errores de servidor.

' DESVENTAJAS

- Añade una dependencia externa ligera al bundle de la aplicación.
- Requiere comprensión de la cadena asíncrona de promesas.

2.3. Proceso de Generación y Compilación de APK (Android Package)

El archivo APK (Android Package) es el contenedor ejecutable comprimido en formato ZIP que agrupa el código compilado, recursos y metadatos de una aplicación Android.

- Anatomía del APK:
 1. AndroidManifest.xml: Archivo binario con permisos (INTERNET), actividades y metadatos.
 2. classes.dex: Bytecode ejecutable para la máquina virtual ART (Dalvik Executable).
 3. resources.arsc: Tabla de recursos precompilados (textos, identificadores, temas).
 4. res/ y assets/: Recursos gráficos y archivos estáticos no estructurados.
 5. lib/: Bibliotecas nativas compiladas en C/C++ organizadas por ABI (arm64-v8a, x86_64).
 6. META-INF/: Bloque de firmas criptográficas y certificados (CERT.RSA, MANIFEST.MF).

2.3.1. Pipeline de Compilación, Debug vs. Release y Firmado Digital

- Pipeline: Código fuente (.kt/.java/.JS) -> Compilador D8/R8 (Desugaring, Ofuscación DEX) -> AAPT2 (Empaquetado de recursos) -> Gradle APK Builder -> zipalign (Alineación a 4 bytes) -> apksigner (Firma criptográfica).
- Debug vs Release: Debug incluye banderas de depuración y firma con debug.keystore genérico. Release aplica ofuscación de código con R8/ProGuard, remueve código muerto y exige firmado con Keystore privado de 2048 bits.
- Zipalign: Alinea todos los archivos no comprimidos en múltiplos de 4 bytes para permitir lectura directa en memoria vía mmap() sin cargar el archivo a la RAM.
- Apksigner: Aplica la firma criptográfica bajo los esquemas v1 (JAR), v2 (APK Block) y v3 (Key Rotation) para garantizar integridad y autenticidad.

' VENTAJAS

- Instalación directa en dispositivos sin depender de tiendas (Sideload).
- Paquete autónomo y portable para auditorías de seguridad.
- Optimización de memoria con zipalign y alta velocidad de verificación con v2.

' DESVENTAJAS

- Mayor peso al empaquetar todas las arquitecturas de CPU (Monolítico).
- Riesgo de ingeniería inversa si no se aplica ofuscación adecuada con R8.

3. DESARROLLO DEL EJEMPLO PRÁCTICO

Se desarrolló una solución móvil completa denominada "MobileStore & API Hub" estructurada por capas (UI Layer, Service Layer, API Client) que implementa Axios para realizar operaciones CRUD completas contra una API REST real, capturar excepciones HTTP y documentar el flujo de construcción de APK.

3.1. Estructura del Código y Cliente Axios Centralizado

El archivo apiClient.js implementa una instancia singleton de Axios con interceptores de Request y Response para inyectar Bearer Tokens, auditar la latencia de red en milisegundos y normalizar errores HTTP:

```
// src/api/apiClient.js - Instancia Centralizada con Interceptores
const axios = require('axios');
const apiClient = axios.create({
  baseURL: 'https://fakestoreapi.com',
  timeout: 10000,
  headers: { 'Content-Type': 'application/json', 'Accept': 'application/json' }
});

// Interceptor de Request: Inyección de Token y Telemetría de Tiempo
apiClient.interceptors.request.use((config) => {
  config.metadata = { startTime: new Date() };
  config.headers['Authorization'] = 'Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...';
  console.log(`[HTTP OUT] ${config.method.toUpperCase()} -> ${config.url}`);
  return config;
});

// Interceptor de Response: Cálculo de Latencia y Mapeo de Errores
apiClient.interceptors.response.use(
  (res) => {
    const latency = new Date() - res.config.metadata.startTime;
    return { success: true, status: res.status, data: res.data, durationMs: latency };
  },
  (err) => {
    const status = err.response ? err.response.status : 'ERR_NETWORK';
    return Promise.reject({ success: false, status, message: err.message });
  }
);
```

3.2. Ejecución y Validación de Pruebas de Integración

Se ejecutó la suite de pruebas test-axios-services.js validando el correcto funcionamiento de los métodos GET, POST, PUT y DELETE en tiempo real:

```
=====
PRUEBAS DE INTEGRACIÓN DE SERVICIOS REST CON AXIOS (EXITOSO)
=====
>>> [TEST 1] GET /products?limit=3    -> 200 OK | Latencia: 843ms | 3 items
>>> [TEST 2] GET /categories          -> 200 OK | Latencia: 781ms | 4 categorías
>>> [TEST 3] POST /products           -> 201 Created | Latencia: 203ms | ID: 21
>>> [TEST 4] PUT /products/1          -> 200 OK | Latencia: 209ms | Título modificado
>>> [TEST 5] DELETE /products/1       -> 200 OK | Latencia: 210ms | Confirmado
=====
```

3.3. Pipeline de Generación, Alineación y Firmado del APK

A través de los scripts build-apk-workflow.sh y generate-keystore.sh se ejecutó la secuencia de construcción y firmado:

```
1. keytool -genkey -v -keystore release-key.jks -alias mobileappkey -keyalg RSA -keysize 2048
2. Ensamblado base: AAPT2 + Dalvik DEX -> dist-apk/app-unsigned-unaligned.apk
3. zipalign -p -f -v 4 app-unsigned-unaligned.apk app-unaligned.apk (Alineación 4 bytes)
4. apksigner sign --ks release-key.jks --out MobileStore-Release-Signed.apk app-unaligned.apk
5. apksigner verify --verbose MobileStore-Release-Signed.apk (Esquemas v1, v2, v3: TRUE)
```

4. CONCLUSIONES

1. Complementariedad Indispensable entre Entornos Físicos y Virtuales:

Los emuladores AVD basados en QEMU y aceleración KVM permiten acelerar radicalmente las fases tempranas de desarrollo e integración continua (CI/CD) sin costo de hardware. Sin embargo, las pruebas en dispositivos físicos continúan siendo insustituibles para auditar con exactitud el consumo de batería, la disipación térmica y la interacción táctil real.

2. Eficiencia y Mantenibilidad de Axios frente a Soluciones Nativas:

Axios demostró ser superior a Fetch para arquitecturas móviles complejas gracias a su sistema de interceptores, que desacopla la inyección de credenciales JWT, el cálculo de latencia de red y el tratamiento homogéneo de respuestas y excepciones en una sola capa centralizada.

3. Resiliencia ante la Inestabilidad de las Redes Móviles:

La configuración de timeouts explícitos y la captura tipada de errores de red en el cliente Axios evitan bloqueos en la interfaz de usuario ante interrupciones de conectividad 4G/5G, garantizando una experiencia de usuario sólida y predecible.

4. Relevancia de Zipalign y el Firmado Criptográfico en Android:

El proceso de alineación a 4 bytes con zipalign optimiza de manera drástica el uso de memoria RAM en el dispositivo móvil mediante mmap(). Asimismo, el firmado criptográfico con apksigner (esquemas v2 y v3) asegura la autenticidad e inviolabilidad del archivo APK antes de su instalación.

5. Calidad del Diseño por Capas en la Solución Práctica:

La separación estricta de responsabilidades entre la capa de interfaz visual (UI), la capa de servicios (Service Layer) y el cliente HTTP permitió implementar un código limpio, modular, fácil de testear y listo para ser empaquetado y desplegado en cualquier entorno Android.

5. BIBLIOGRAFÍA (NORMAS APA 7ma EDICIÓN)

- Android Open Source Project. (2024). Android Debug Bridge (adb) architecture and command reference. Android Developers. <https://developer.android.com/tools/adb>
- Axios Community. (2024). Axios HTTP client documentation: Interceptors, instances and error handling. Axios HTTP. <https://axios-http.com/docs/intro>
- Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures (Doctoral dissertation, University of California, Irvine).
- Google Developers. (2024). Run apps on the Android Emulator and manage Android Virtual Devices (AVD). Android Studio Documentation. <https://developer.android.com/studio/run/emulator>
- Google Developers. (2024). Sign your app: APK Signature Scheme v2, v3 and keytool management. Android Developers Guide. <https://developer.android.com/studio/publish/app-signing>
- Google Developers. (2024). Reduce app size and optimize alignment with zipalign. Android Studio Tools. <https://developer.android.com/tools/zipalign>
- IETF (Internet Engineering Task Force). (2014). Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content (RFC 7231). <https://datatracker.ietf.org/doc/html/rfc7231>
- Martin, R. C. (2018). Clean Architecture: A craftsman's guide to software structure and design. Prentice Hall.
- Meier, R., & Lake, I. (2018). Professional Android (4th ed.). Wrox - John Wiley & Sons.
- Smyth, N. (2023). Android Studio Panda Essentials - Kotlin Edition: Developing Android Apps Using Android Studio and Kotlin. Payload Media.

